



VINUNIVERSITY

Lab 08: Priority Queues and Heaps.

Week 04, May 04 2026

**College of Engineering and Computer Science,
Object-Oriented Programming & Data Structures Spring**

Lab Practice Submission Instructions:

- This is an individual lab practice and will typically be assigned in the computer laboratory.
- Your program should work correctly on all inputs. If there are any specifications about how the program should be written (or how the output should appear), those specifications should be followed.
- Your code and functions/modules should be appropriately commented. However, try to avoid making your code overly busy (e.g., include a comment on every line).
- Variables and functions should have meaningful names, and code should be organized into functions/methods where appropriate.
- Academic honesty is required in all work you submit to be graded. You should **NOT** copy or share your code with other students to avoid plagiarism issues.
- Use the template provided to prepare your solutions (modified to provide your student ID, your name, etc).
- You should **upload your .java file(s) to Codeforces before the end of the laboratory session** unless the instructor gave a specified deadline. **No Canvas or Codeforces submission** will get **0** point.
- Submit a separate .java file for each lab problem, with the exact filename specified in each lab question.
- Late submission of lab practice without an approved extension will incur the following penalties:
 1. No submission by the deadline will incur 0.25 point deduction for each problem (most of the problems are due at the end of the lab session).
 2. The instructor will deduct an additional 0.25 point per problem for each day past the deadline.
 3. The penalty will be deducted until the maximum possible score for the lab practice reaches zero (0%) unless otherwise specified by the instructor.
- **Navigate to the Lab:** Open the specific link provided for this week's lab: <https://vinuni26oop.contest.codeforces.com/>.

Problem 1: Prelab - Heap Array Indices

Write a Java program that calculates the indices of the parent, left child, and right child for a given node index i in a 0-based array representation of a binary heap of size N .

As discussed in the lecture, for a zero-based index array, if a node is at index i , its left child is at $2i + 1$, its right child is at $2i + 2$, and its parent is at $(i - 1)/2$. If the node is the root, output -1 for the parent. If a child index is $\geq N$, output -1 to indicate it does not exist.

Input

Two space-separated integers: N (size of the heap) and i (the target node index).

Output

Three space-separated integers representing the indices of the parent, left child, and right child.

Examples

Test Case 1:

Input:

5 1

Output:

0 3 4

Test Case 2:

Input:

5 2

Output:

0 -1 -1

Problem 2: Compulsory - MaxHeap Construction

Write a Java program to build a **MaxHeap** from an empty array by inserting elements one by one.

When inserting an element, place it at the end of the complete binary tree (the next available index) and "bubble up" by comparing it with its parent. Swap them if the newly inserted element is strictly greater, and repeat until the MaxHeap property is satisfied.

Input

The first line contains an integer N , the number of elements to insert. The second line contains N space-separated integers.

Output

The array representation of the MaxHeap after all elements are inserted, separated by spaces.

Examples

Test Case 1:

Input:

5

10 5 20 1 30

Output:

30 20 10 1 5

Test Case 2:

Input:

4

3 9 2 1

Output:

9 3 2 1

Problem 3: Compulsory - HeapSort (Ascending)

Write a Java program to sort an array of integers in **ascending** order using the HeapSort algorithm.

HeapSort requires converting the unsorted array into a MaxHeap first. Then, repeatedly extract the maximum element (swapping the root with the last element), reduce the heap size by 1, and run a heapify-down operation on the root.

Input

The first line contains an integer N . The second line contains N space-separated integers.

Output

The sorted array in ascending order, separated by spaces.

Examples

Test Case 1:

Input:

5

4 10 3 5 1

Output:

1 3 4 5 10

Test Case 2:

Input:

6

12 11 13 5 6 7

Output:

5 6 7 11 12 13

Problem 4: Compulsory - K-th Largest Element

Write a Java program to find the **K-th largest** element in a stream of numbers.

To optimize performance, do **not** sort the entire array. Use Java's `PriorityQueue` (which is a MinHeap by default) to keep track of the K largest elements seen so far. If the size of the heap exceeds K , remove the minimum element. At the end of the loop, the root of the heap is your answer.

Input

The first line contains two integers N (number of elements) and K . The second line contains N space-separated integers.

Output

A single integer representing the K-th largest element.

Examples

Test Case 1:

Input:

6 2

3 2 1 5 6 4

Output:

5

Test Case 2:

Input:

4 1

10 20 30 40

Output:

40

Problem 5: Advanced - Top K Frequent Integers

Write a Java program to find the K most frequent integers in an array.

You should first use a `HashMap` to count the occurrences of each number. Then, push the numbers into a `PriorityQueue` (configured as a `MinHeap` using a custom `Comparator` based on frequencies). Limit the heap size to K .

Input

The first line contains two integers N (number of elements) and K . The second line contains N space-separated integers.

Output

The K most frequent elements, separated by a space.

Examples

Test Case 1:

Input:

6 2

1 1 1 2 2 3

Output:

1 2

Test Case 2:

Input:

4 1

5 5 5 2

Output:

5